

ANAL KUMAR BISWAS

Senior Software Engineer | Distributed Systems, Cloud Architecture & Applied AI

React | Node.js | Python | Java | Angular | AWS | GCP | Kubernetes | Microservices | Event-Driven Architecture | LLM & RAG

Email: kumaranal.official@gmail.com | Phone: +91 7003080703 | Location: Kolkata, India (Open to Relocation)

LinkedIn: linkedin.com/in/anal-kumar-biswas-733488356 | Portfolio: analkumarbiswas.com

PROFESSIONAL SUMMARY

Senior Software Engineer with 6 years owning end-to-end architecture (HLD/LLD) for distributed, event-driven systems and production LLM/RAG platforms on AWS, GCP and Kubernetes, across automotive, telecom, healthcare, sports and utility domains.

Cut production MTTR by 40% (~5 hrs to ~3 hrs) across ~1M daily events and release time from 45 to 10 minutes; seeking a senior backend / applied-AI engineering role with ownership of system design end to end.

ARCHITECTURE & TECHNICAL LEADERSHIP

- System design ownership:** Author HLD and LLD for greenfield and decomposition work — component and sequence diagrams, API contracts, event schemas, data models and written decision records reviewed with stakeholders before implementation.also having concepts of Data Structures & Algorithms .
- Technology selection with documented tradeoffs:** Kafka vs. RabbitMQ, SFU vs. peer-to-peer WebRTC, GraphQL vs. REST, algorithmic optimization vs. vertical scaling — each chosen against throughput, ordering, cost and operational load, not novelty.
- Cross-team delivery:** Set service standards for observability, idempotency, security and CI/CD on teams of 4–14; mentor engineers through design review, code review and pairing.

TECHNICAL SKILLS

Programming Languages: Python, JavaScript (ES6+), TypeScript, Java, SQL, C++

Frontend & Frameworks: React.js, Next.js, Angular, Electron, HTML5, CSS3, GraphQL

Backend & Architecture: Node.js, NestJS, Express.js, Python FastAPI, Core Java, Spring Boot, RESTful APIs, Microservices Architecture, Event-Driven Architecture, Kafka, RabbitMQ, WebRTC, System Design, High-Level Design (HLD), Low-Level Design (LLD), Scalability, Data Structures & Algorithms

Cloud, DevOps & Infrastructure: AWS (EKS, ECS, EC2, S3, Lambda, Lightsail), Google Cloud Platform (GCP), Kubernetes (On-Prem / Bare-Metal), Docker, Helm, Terraform, Jenkins, CI/CD Pipelines, Git

Databases & AI Tools: PostgreSQL, MongoDB, Redis, OpenSearch, Supabase, Prisma, Sequelize | LLMs, RAG, LangChain, Model Context Protocol (MCP), Claude Code

Security & Observability: Keycloak, OAuth 2.0, OpenID Connect, RBAC, Microsoft Entra ID, OpenTelemetry, Distributed Tracing, Swagger, Jest, Code Review, Agile / Scrum

PROFESSIONAL EXPERIENCE

GNAPI TECHNOLOGIES PVT. LTD. | India

Senior Software Engineer | Jul 2025 – Present

Project 1: AI-Assisted Live Broadcasting Platform (built from scratch) | Team of 8 — owned system architecture (HLD/LLD) and the media orchestration services

Business Need: Legacy streaming buffered past ~2,000 concurrent viewers and required a producer on site for every match.

- Architected a greenfield WebRTC and LiveKit streaming platform from the ground up — component and sequence diagrams, API contracts and data model — replacing the legacy buffering pipeline entirely.
- Engineered a React producer console and Python FastAPI media orchestration services integrated with LangChain for automated match summaries, using Redis caching and fallback paths for fault tolerance.
- Deployed on Amazon EKS with Horizontal Pod Autoscaling and bare-metal GPU nodes, scaling from ~2,000 to 10,000+ concurrent viewers across 50+ camera feeds at 99.9% availability measured per match window across the season.
- Design tradeoff:** chose a LiveKit SFU over a peer-to-peer mesh because mesh publisher bandwidth grew linearly with every viewer; the SFU kept publisher upload flat and made server-side GPU transcoding and recording possible, accepting the cost of operating and scaling our own media plane.

Tech & Tools: React, TypeScript, Python, FastAPI, WebRTC, LiveKit, LangChain, Redis, PostgreSQL, Docker, Kubernetes, Helm, Terraform, AWS (EKS, EC2, S3), bare-metal GPU nodes, Jenkins

Project 2: Geospatial Asset Management Platform with RAG Assistant (IQGeo) | Team of 14 — owned the RAG pipeline and multi-cloud provisioning

Business Need: Field engineers managing 100,000+ electrical assets spent 30–45 minutes searching thousands of pages of standards and manuals before acting on site.

- Developed a React and Python FastAPI platform with MongoDB and GeoServer enabling lifecycle tracking and spatial querying across 100,000+ electrical assets.
- Built an end-to-end RAG pipeline using LangChain and OpenSearch vector indexing that cut a typical manual lookup from 30–45 minutes to under 1 minute, returning citation-backed answers engineers could verify on site.
- Automated multi-target provisioning via Terraform across GCP, AWS EKS and on-premise bare-metal Kubernetes clusters to satisfy client data-residency and compliance policies from a single pipeline.

Tech & Tools: React, Python, FastAPI, LangChain, OpenSearch, MongoDB, PostgreSQL, GeoServer, Redis, AWS (EKS, S3), GCP, bare-metal Kubernetes, Docker, Helm, Terraform, Jenkins

Project 3: Event-Driven QA Automation Platform | Team of 6 — owned event schema design, service LLD and SSO integration

Business Need: QA teams triggered test suites by hand and raised Jira defects manually, with no single view of what failed or why.

- Decomposed a monolithic runner into 11 NestJS microservices over Kafka — chosen over RabbitMQ for replayable history and per-run ordering — defining event schemas, LLD specs and a React analytics dashboard.
- Implemented idempotency keys, dead-letter queues and OpenTelemetry distributed tracing across ~1M daily events, reducing production MTTR by 40% (from ~5 hours to ~3 hours).

- Configured Keycloak SSO with Microsoft Entra ID for 5,000+ users across 12 applications; built AI developer agents (MCP, Claude Code) that cut new-service setup from 4 hours to 15 minutes.

Tech & Tools: React, Node.js, NestJS, TypeScript, Kafka, Redis, PostgreSQL, OpenSearch, Python, MCP, Claude Code, Keycloak, OAuth 2.0, OpenID Connect, Microsoft Entra ID, Docker, Kubernetes, Helm, AWS (EKS, S3), Jenkins, OpenTelemetry

XOTIV TECHNOLOGIES PVT. LTD. | India

Full Stack Developer | Dec 2023 – Jun 2025

Project 1: Golf Course & Tournament Management Platform | Team of 6 — owned API architecture and the AWS migration

Business Need: Clubs ran tee-time bookings, memberships, handicaps and tournament draws on spreadsheets; the company wanted one product to sell to many clubs.

- Architected Angular frontend portals and Node.js/Express backend APIs over PostgreSQL for a multi-tenant product serving several clubs from one codebase.
- Engineered complex business logic for draw generation, live leaderboards and real-time scorecard reconciliation.
- Containerized the application stack and migrated infrastructure to AWS ECS with Jenkins CI/CD pipelines, cutting release deployment time from 45 minutes to 10 minutes.
- Design tradeoff:** introduced GraphQL over REST for the club portals because every club dashboard needed a different slice of the same tee-sheet, member and scoring data, and REST was forcing either over-fetching or endpoint sprawl; kept REST for webhooks and third-party integrations, where HTTP caching and simplicity mattered more than query flexibility.

Tech & Tools: Angular, TypeScript, Node.js, Express.js, GraphQL, Sequelize, PostgreSQL, Redis, Docker, AWS (ECS, EC2, S3), Jenkins, Jest, Jira

Project 2: DEVEBOL – AI Sports Education Platform | Team of 4 — owned the Python AI service and the notification pipeline

Business Need: Coaches and athletes needed drill recommendations and rule clarifications during live sessions, not a search through a video library and rulebook.

- Designed a React/Next.js and Node.js product interface while isolating AI capabilities into an autonomous Python FastAPI service, so model and prompt changes shipped without redeploying the product.
- Implemented LangChain RAG with OpenSearch vector search, caching embeddings and frequent queries in Redis, lowering p95 API latency from ~850 ms to ~550 ms while sustaining ~500 RPS during live sports events on AWS EKS with autoscaling.
- Built the transactional notification pipeline on Twilio and SendGrid for session reminders, drill assignments and coach feedback — templated messages, delivery-status webhooks and retry with exponential backoff on provider failure — replacing manual coach follow-up.

Tech & Tools: React, Next.js, Node.js, Python, FastAPI, LangChain, OpenSearch, Prisma, PostgreSQL, Supabase, Redis, Firebase, Twilio, SendGrid, Docker, Kubernetes, AWS (EKS, S3, Lambda), Jenkins

Project 3: ReadMyRhythm – Telemedicine & EKG Review Platform | Team of 4 — owned PHI compliance design and the billing integration

Business Need: Cardiology groups exchanged institutional EKG studies over email with no audit trail, and reconciled billing by hand each month.

- Created a React/Next.js clinician platform with Python FastAPI EKG file parsing and an LLM drafting workflow that produced first-pass reports for cardiologist review and sign-off.
- Enforced strict PHI compliance using TrueVault, Supabase and short-lived signed URLs, replacing email exchange with a fully audited access trail and completing audits with zero compliance findings.
- Integrated the Stripe API to reconcile ~10,000 monthly institutional transactions automatically, eliminating a manual month-end reconciliation; deployed on AWS ECS with private subnets.

Tech & Tools: React, Next.js, Node.js, Python, FastAPI, Supabase, PostgreSQL, TrueVault, Stripe API, Docker, AWS (ECS, EC2, S3), Jenkins, Jest

TATA CONSULTANCY SERVICES (TCS) | India

Software Analyst | Nov 2020 – Nov 2023

Project 1: Dealer Incentive Management System (FIAT Automobile) | Owned LLD for settlement services under the lead architect

Business Need: Dealer incentives and commissions were calculated in spreadsheets, making mid-quarter scheme changes slow and disputes frequent.

- Developed Spring Boot microservices and Angular dashboards for automotive incentive processing, authoring the low-level design for settlement services under the lead architect.
- Optimized batch processing from 3 hours to 40 minutes for a system processing \$50M+ in annual incentives; automated AWS EC2 releases via Jenkins.
- Design tradeoff:** met the batch window by partitioning dealer data and replacing per-record queries with set-based aggregation rather than moving to a larger EC2 instance — roughly 4x headroom at no added infrastructure cost, at the price of re-verifying settlement totals against a quarter of historical runs before cutover.

Tech & Tools: Java, Spring Boot, REST APIs, Microservices, Angular, TypeScript, PostgreSQL, Jenkins, Git, AWS EC2, Swagger, Jira

Project 2: Optical Fiber Network Management System (Comcast) | Owned the network topology APIs and dashboard query layer

Business Need: Field and NOC teams had no single view of fiber routes, splice points and maintenance windows, and traffic reports arrived a day late.

- Created Angular fiber network management views and Node.js REST services over MongoDB for real-time network topology tracking.
- Redesigned complex MongoDB aggregation pipelines to bring dashboard queries from repeated 30-second timeouts down to ~2 seconds (60%+ latency reduction on the queries that did complete); containerized builds using Docker and Jenkins.

Tech & Tools: Angular, TypeScript, Node.js, Express.js, MongoDB, Redis, Docker, Jenkins, Git, Jest, Jira

EDUCATION

Bachelor of Technology (B.Tech), Electrical Engineering

Academy of Technology (West Bengal University of Technology), West Bengal | 2016 – 2020 | CGPA: 8.1